

Week 2 Day 4 Hands-on Lab: Runtime Config and Observability

이 lab은 Compose를 사용하지 않는다. 같은 image라도 실행 옵션이 달라지면 상태가 달라진다는 것을 docker run, logs, inspect, exec, stats, failure drill로 확인한다.

Phase A: preflight

```
cd /mnt/d/paperclip
docker version
docker image ls nginx
docker image ls postgres
```

Expected:

Docker client/server 정보가 출력된다.
nginx 또는 postgres image가 없으면 이후 run 단계에서 pull된다.

Phase B: env file 준비

```
cd /mnt/d/paperclip
mkdir -p week2/day4/labs/env-report
sed -n '1,120p' week2/day4/labs/env-report/.env.example
cp week2/day4/labs/env-report/.env.example week2/day4/labs/env-report/.env
sed -n '1,120p' week2/day4/labs/env-report/report.sh
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
```

주의: DB_PASSWORD 값은 실습용 placeholder다. 실제 개인 password, cloud key, Docker Hub token을 넣지 않는다.

.env는 다음처럼 한 줄에 하나씩 기록한다.

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
# DEBUG=true
```

env file은 container 실행 시점에 다음처럼 로드한다.

```
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 env
```

파일 형식은 다음 규칙을 따른다.

형식	의미
KEY=value	container 안에 KEY=value로 주입
KEY	host shell에 export된 같은 이름의 값을 사용
# comment	comment로 무시

이미 실행된 container는 env file을 다시 읽지 않는다. .env를 고쳤다면 container를 새로 만들어야 한다.

환경별 파일을 나누는 패턴도 확인한다.

```
cat > week2/day4/labs/env-report/.env.dev <<'EOF'
APP_ENV=dev
FEATURE_FLAG=on
API_BASE_URL=http://localhost:3000
EOF

cat > week2/day4/labs/env-report/.env.staging <<'EOF'
APP_ENV=staging
FEATURE_FLAG=on
API_BASE_URL=https://staging.example.com
EOF

cat > week2/day4/labs/env-report/.env.prod <<'EOF'
APP_ENV=prod
FEATURE_FLAG=off
API_BASE_URL=https://api.example.com
EOF

docker run --rm --env-file week2/day4/labs/env-report/.env.dev alpine:3.20 sh
-c 'echo "$APP_ENV $FEATURE_FLAG $API_BASE_URL"'
```

```
docker run --rm --env-file week2/day4/labs/env-report/.env.staging alpine:3.20
sh -c 'echo "$APP_ENV $FEATURE_FLAG $API_BASE_URL"'
docker run --rm --env-file week2/day4/labs/env-report/.env.prod alpine:3.20 sh
-c 'echo "$APP_ENV $FEATURE_FLAG $API_BASE_URL"'
```

Expected:

```
dev on http://localhost:3000
staging on https://staging.example.com
prod off https://api.example.com
```

해석: image는 같고 env file만 바꿔 환경 차이를 표현한다. 이 패턴은 나중에 Kubernetes의 ConfigMap/Secret, Terraform의 dev.tfvars, staging.tfvars, prod.tfvars로 이어진다.

Phase C: -e와 --env-file 비교

```
docker run --rm -e APP_ENV=inline -e FEATURE_FLAG=off alpine:3.20 env | sort |
grep -E 'APP_ENV|FEATURE_FLAG'
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 env \
| sort \
| grep -E 'APP_ENV|FEATURE_FLAG|DB_PASSWORD' \
| sed -E 's/DB_PASSWORD=.*DB_PASSWORD=***masked***/'
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 sh -c
'echo "APP_ENV=$APP_ENV FEATURE_FLAG=$FEATURE_FLAG DB_PASSWORD=$DB_PASSWORD"'
\
| sed -E 's/DB_PASSWORD=.*DB_PASSWORD=***masked***/'
```

Expected:

```
APP_ENV=inline
FEATURE_FLAG=off
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
APP_ENV=practice FEATURE_FLAG=on DB_PASSWORD=***masked***
```

해석: -e는 명령줄에 직접 들어가고, --env-file은 파일에서 읽는다. 둘 다 runtime config이며 image를 새로 build하지 않는다.

비교	-e	--env-file
입력 위치	실행 명령 안	별도 파일
적합한 경우	한두 개 값을 임시로 바꿀 때	여러 값을 재사용할 때
기록 위험	shell history에 값이 직접 남기 쉬움	파일 관리가 필요함
공유 방식	명령 예시만 공유	.env.example만 공유
적용 시점	container 생성 시점	container 생성 시점

실제 기록에는 password 값을 그대로 남기지 않는다.

```
DB_PASSWORD=***masked***
```

Phase D: nginx run/logs/HTTP

```
docker rm -f paperclip-day4-nginx || true
docker run -d --name paperclip-day4-nginx -p 18084:80 nginx:1.27-alpine
docker ps --filter name=paperclip-day4-nginx
docker logs paperclip-day4-nginx --tail 30
curl -I http://localhost:18084
```

Expected:

```
STATUS Up
0.0.0.0:18084->80/tcp
HTTP/1.1 200 OK
```

해석: Up은 process 상태이고, HTTP/1.1 200 OK는 서비스 관점의 확인이다.

포트가 이미 사용 중이면 다음처럼 점유 주체를 먼저 확인한다.

```
docker ps --format 'table {{.Names}}\t{{.Ports}}' | grep 18084 || true
ss -ltnp | grep 18084 || true
```

다른 실습 container가 점유 중이면 정리하고, 다른 서비스가 점유 중이면 host port를 18085:80처럼 바꿔 실행한다.

환경 설정을 로그로 남기는 경우에는 secret masking을 확인한다.

```
docker rm -f paperclip-day4-log-env || true
docker run --name paperclip-day4-log-env --env-file week2/day4/labs/env-report/.env -v "$PWD/week2/day4/labs/env-report:/work:ro" alpine:3.20
```

```
/work/report.sh || true
docker logs paperclip-day4-log-env
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
```

Phase E: inspect와 exec

```
docker inspect paperclip-day4-nginx --format '{{json
.NetworkSettings.Ports}}'
docker inspect paperclip-day4-nginx --format 'Image={{.Config.Image}} Restart=
{{json .HostConfig.RestartPolicy}}'
docker exec paperclip-day4-nginx ls -l /usr/share/nginx/html
docker exec paperclip-day4-nginx sh -c 'ps | head'
docker exec paperclip-day4-nginx sh -c 'cat /etc/nginx/conf.d/default.conf |
sed -n "1,40p"'
docker rm -f paperclip-day4-env-inspect || true
docker run -d --name paperclip-day4-env-inspect --env-file
week2/day4/labs/env-report/.env alpine:3.20 sleep 300
docker inspect paperclip-day4-env-inspect --format '{{range .Config.Env}}
{{println .}}{{end}}' \
  | grep -E 'APP_ENV|FEATURE_FLAG|DB_PASSWORD' \
  | sed -E 's/DB_PASSWORD=.*\/DB_PASSWORD=***masked***/'
docker exec paperclip-day4-env-inspect sh -c 'env | grep -E
"APP_ENV|FEATURE_FLAG|DB_PASSWORD"' \
  | sed -E 's/DB_PASSWORD=.*\/DB_PASSWORD=***masked***/'
```

Expected:

```
Ports={"80/tcp":[{"HostIp":"0.0.0.0","HostPort":"18084"}]}
Image=nginx:1.27-alpine
index.html
nginx
server {
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
```

해석: inspect/exec는 env 값을 보여줄 수 있다. 실습 중 실제 값을 확인할 수는 있지만 제출물과 screenshot에는 masking된 출력만 남긴다.

exec shell read-only 확인

여러 파일과 process를 이어서 볼 때는 shell로 들어갈 수 있다. 단, 이 shell은 상태 확인용이다. container 안에서 파일을 고치거나 package를 설치하지 않는다.

```
docker exec -it paperclip-day4-nginx sh
```

container 안에서 실행한다.

```
pwd
whoami
ls -al /usr/share/nginx/html
cat /usr/share/nginx/html/index.html | head
cat /etc/nginx/conf.d/default.conf | sed -n '1,40p'
ps
exit
```

Expected:

```
/
root
index.html
server {
PID    USER
```

금지 예시는 다음과 같다.

금지 명령/행동	이유
vi, nano, sed -i로 설정 파일 수정	image/Dockerfile/compose에 변경 근거가 남지 않음
rm, mv, cp로 내부 파일 변경	재시작/재생성 시 사라지거나 원인 추적이 어려움
apk add, apt install로 도구 설치	container를 수동 snowflake 상태로 만듦
shell에서 hotfix 후 정상이라고 보고	같은 image로 재배포하면 문제가 되살아남

수정이 필요하면 shell을 빠져나와 Dockerfile, env file, bind mount 원본 파일, docker run option 또는 Day 5의 compose.yaml에 반영한다.

Phase F: stats와 restart policy

```
docker stats paperclip-day4-nginx --no-stream
docker inspect paperclip-day4-nginx --format 'before={{json
.HostConfig.RestartPolicy}}'
docker update --restart unless-stopped paperclip-day4-nginx
docker inspect paperclip-day4-nginx --format 'after={{json
.HostConfig.RestartPolicy}}'
docker rm -f paperclip-day4-restart-missing-env || true
docker run -d --name paperclip-day4-restart-missing-env --restart on-failure:2
postgres:16-alpine
sleep 3
docker inspect paperclip-day4-restart-missing-env --format 'RestartCount=
{{.RestartCount}} Status={{.State.Status}} ExitCode={{.State.ExitCode}}'
docker logs paperclip-day4-restart-missing-env --tail 20 || true
```

Expected:

```
NAME                CPU %      MEM USAGE / LIMIT
before={"Name":"no","MaximumRetryCount":0}
after={"Name":"unless-stopped","MaximumRetryCount":0}
RestartCount=2
POSTGRES_PASSWORD
```

해석: restart policy는 process를 다시 띄우려는 정책이지 원인을 고치는 기능이 아니다.

Phase G: failure drill 1 - missing env

```
docker rm -f paperclip-day4-pg-missing-env || true
docker run --name paperclip-day4-pg-missing-env postgres:16-alpine || true
docker ps -a --filter name=paperclip-day4-pg-missing-env
docker logs paperclip-day4-pg-missing-env --tail 40 || true
```

Expected failure output:

```
Error: Database is uninitialized and superuser password is not specified.
You must specify POSTGRES_PASSWORD to a non-empty value
STATUS Exited
```

Hint:

postgres image는 최초 DB 초기화에 POSTGRES_PASSWORD가 필요하다.
container가 Exited라면 docker logs가 첫 확인 명령이다.

Fix and recheck:

```
docker rm -f paperclip-day4-pg-missing-env || true
docker run -d --name paperclip-day4-pg-ok -e POSTGRES_PASSWORD=practice-only
postgres:16-alpine
docker logs paperclip-day4-pg-ok --tail 40
```

Expected recovery:

```
database system is ready to accept connections
```

Wrong env file path:

```
docker run --rm --env-file week2/day4/labs/env-report/.env.production
alpine:3.20 env || true
ls week2/day4/labs/env-report
```

Expected failure output:

```
no such file or directory
```

Hint:

실행 전에 env file 경로가 틀린 문제다. app이나 image 문제가 아니다.

Phase H: failure drill 2 - wrong port

```
curl -I http://localhost:80 || true
curl -I http://localhost:18084 || true
docker ps --filter name=paperclip-day4-nginx
```

Expected failure output:

```
curl: (7) Failed to connect to localhost port 80
HTTP/1.1 200 OK
0.0.0.0:18084->80/tcp
```

Hint:

host는 18084로 접근한다. container 내부 80은 host 80을 자동으로 열지 않는다.

Phase I: failure drill 3 - wrong network

```
docker rm -f paperclip-day4-net-web || true
docker network rm paperclip-day4-net-a paperclip-day4-net-b || true
docker network create paperclip-day4-net-a
docker network create paperclip-day4-net-b
docker run -d --name paperclip-day4-net-web --network paperclip-day4-net-a
nginx:1.27-alpine
docker run --rm --network paperclip-day4-net-b alpine:3.20 wget -S -O-
http://paperclip-day4-net-web || true
```

Expected failure output:

```
bad address 'paperclip-day4-net-web'
```

Fix and recheck:

```
docker run --rm --network paperclip-day4-net-a alpine:3.20 wget -S -O-
http://paperclip-day4-net-web | head
```

Expected recovery:

```
HTTP/1.1 200 OK
```

Phase J: failure drill 4 - stale volume

```
docker rm -f paperclip-day4-pg-volume || true
docker volume rm paperclip-day4-pgdata || true
docker run -d --name paperclip-day4-pg-volume -e POSTGRES_PASSWORD=practice-
only -e POSTGRES_DB=first -v paperclip-day4-pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5
docker rm -f paperclip-day4-pg-volume
docker run -d --name paperclip-day4-pg-volume -e POSTGRES_PASSWORD=practice-
only -e POSTGRES_DB=second -v paperclip-day4-pgdata:/var/lib/postgresql/data
postgres:16-alpine
```

```
docker logs paperclip-day4-pg-volume --tail 30
docker volume inspect paperclip-day4-pgdata
```

Expected signal:

Database directory appears to contain a database; Skipping initialization

Hint:

env를 바꿔도 기존 volume data가 남아 있으면 최초 초기화가 다시 실행되지 않는다.
volume 삭제는 data reset이므로 의도적으로 판단해야 한다.

Phase K: cleanup/security audit

```
docker ps -a --filter name=paperclip-day4
docker network ls | grep paperclip-day4 || true
docker volume ls | grep paperclip-day4 || true
docker system df
```

Cleanup:

```
docker rm -f paperclip-day4-nginx paperclip-day4-log-env paperclip-day4-env-
inspect paperclip-day4-pg-ok paperclip-day4-pg-missing-env paperclip-day4-net-
web paperclip-day4-restart-missing-env paperclip-day4-pg-volume || true
docker network rm paperclip-day4-net-a paperclip-day4-net-b || true
rm -f week2/day4/labs/env-report/.env week2/day4/labs/env-report/.env.dev
week2/day4/labs/env-report/.env.staging week2/day4/labs/env-report/.env.prod
# data reset이 필요할 때만 실행
# docker volume rm paperclip-day4-pgdata
```

Audit note:

삭제한 것:
남긴 것:
volume을 삭제하지 않은 이유:
secret 값이 남은 출력/screenshot 여부:

Phase L: Compose mapping preview

지금 실행한 option	Day 5에서 옮길 Compose 항목
-p 18084:80	services.web.ports
-e POSTGRES_PASSWORD=...	services.db.environment
--env-file .env	env_file 또는 \${VARIABLE}
.env.dev/.env.staging/.env.prod	환경별 env file
--network paperclip-day4-net-a	networks
docker logs <name>	docker compose logs <service>
docker exec <name> ...	docker compose exec <service> ...

Phase M: final evidence handoff

다음 표를 채우지 못하면 Day 4를 완료한 것이 아니다. 명령을 실행한 횟수가 아니라, 어떤 증거로 어떤 판단을 했는지가 완료 기준이다.

항목	내가 남길 증거	판단
env 주입	-e 또는 --env-file 실행 결과	image rebuild 없이 runtime config가 바뀌었다.
secret masking	DB_PASSWORD=***masked*** 출력	실제 secret 값을 기록하지 않았다.
HTTP 확인	curl -I http://localhost:18084	Up과 서비스 정상 응답을 구분했다.
logs	docker logs ... --tail	startup/access/error 중 무엇을 봤는지 적는다.
inspect	port/env/restart/mount 중 선택 field	전체 JSON이 아니라 필요한 field만 남겼다.
exec	filesystem/process/env 중 하나	container 내부 관찰 결과를 남겼다.
exec shell safety	read-only 확인과 금지 행동	shell에서 수정하지 않고 source로 돌아갈 기준을 적었다.
restart	RestartCount, ExitCode	restart policy가 원인을 고치지 못한다는 것을 설명했다.
failure RCA	실패 출력 한 줄	config/port/network/volume/image 중 하나로 분류했다.
cleanup	삭제/보존 목록	volume 삭제가 data reset인지 판단했다.
Day 5 mapping	docker run option -> Compose field	Compose로 옮길 이유를 설명했다.

RCA note:

Symptom:

First evidence command:

Key output line:

Category: env / port / network / volume / image / process
 Fix:
 Recheck:
 Data deletion risk:

Phase N: 8교시 | Prometheus/Grafana preview

이 phase는 Day 4의 마무리 장면이다. docker logs와 docker stats로 본 정보를 Compose 기반 observability stack에서 어떻게 보는지 미리 체험한다.

```
cd /mnt/d/paperclip/week2/day4/labs/observability-preview
export DOCKER_ROOT_DIR="$(docker info --format '{{.DockerRootDir}}')"
docker compose config
docker compose --profile load config
docker compose up -d
docker compose ps
for i in 1 2 3 4 5; do curl -I http://localhost:18085 || true; sleep 1; done
docker compose logs sample-web --tail 20
docker compose logs log-generator --tail 20
```

Expected:

```
HTTP/1.1 200 OK
level=info service=log-generator event=heartbeat
```

Open:

```
Grafana:    http://localhost:13000  admin / practice-only
Prometheus: http://localhost:19090
cAdvisor:   http://localhost:18086
```

Grafana Explore에서 확인한다.

Source	Query	의미
Prometheus	up	scrape 대상이 살아 있는지 확인
Prometheus	container_memory_usage_bytes	container memory metrics 확인
Loki	{job="docker"}	Docker container log 확인

Loki를 curl로 검증할 때는 instant query가 아니라 range query를 사용한다.

WSL/Linux:

```
now_ns=$(date +%s%N)
start_ns=$((now_ns - 300000000000))

curl -G -s 'http://localhost:13100/loki/api/v1/query_range' \
  --data-urlencode 'query={job="docker"}' \
  --data-urlencode "start=$start_ns" \
  --data-urlencode "end=$now_ns" \
  --data-urlencode 'limit=5'
```

macOS:

```
end_time=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
start_time=$(date -u -v-5M +"%Y-%m-%dT%H:%M:%SZ")

curl -G -s 'http://localhost:13100/loki/api/v1/query_range' \
  --data-urlencode 'query={job="docker"}' \
  --data-urlencode "start=$start_time" \
  --data-urlencode "end=$end_time" \
  --data-urlencode 'limit=5'
```

Impact drill:

```
docker compose --profile load up -d cpu-spike
docker stats --no-stream
sleep 20
```

Grafana Explore > Prometheus:

```
rate(container_cpu_usage_seconds_total[1m])
```

해석:

docker stats는 현재 순간값이다.
Prometheus/Grafana는 시간에 따른 변화를 보여준다.
CPU spike가 보이면 docker logs/compose logs로 어떤 container가 무엇을 했는지 다시
쭈른다.

```
docker compose stop cpu-spike
```

환경에 따라 Docker Desktop/WSL에서 Promtail이 /var/lib/docker/containers를 읽지 못할 수
있다. 이 경우에도 docker compose logs로 일반 로그를 확인하고, Prometheus/cAdvisor metrics 확

인까지를 preview 성공으로 본다.

Troubleshooting:

환경	오류/증상	힌트
WSL/Linux	docker-credential-desktop.exe not found	DOCKER_CONFIG 임시 디렉터리에 빈 config.json을 두고 실행
WSL/Linux	cAdvisor가 /var/lib/docker를 못 읽음	export DOCKER_ROOT_DIR="\$(docker info --format '{{.DockerRootDir}}')\" 후 재실행
WSL/Linux/macOS	port is already allocated	docker ps로 점유 port 확인. 이 lab은 cAdvisor 18086 사용
WSL/Linux/macOS	Loki instant query 오류	/loki/api/v1/query_range 사용
macOS	date +%s%N이 nanosecond로 안 나옴	macOS용 date -u -v -5M 예시 사용
macOS	Promtail/Loki log가 비어 있음	Docker Desktop VM log path 제한 가능. docker compose logs로 기본 로그 확인
macOS	cAdvisor device/mount 오류	Docker Desktop 제약 가능. docker stats와 Prometheus/Grafana UI 확인 후 실패 원인을 토론

Cleanup:

```
docker compose down
# dashboard/log data까지 reset할 때만
# docker compose down -v
```